QuickCart Black-Box Testing Report

- Roll Number: `2024101096`
- Base URL: `http://localhost:8080/api/v1`
- Tools: `pytest`, `requests`, `coverage`

1. Test Case Design

I designed test cases from the API documentation only (black-box approach), covering:

- Valid requests
- Invalid inputs
- Missing fields/headers
- Wrong data types
- Boundary values

1.1 Implemented Automated Test Modules

- test_01_admin.py
- test_02_profile.py
- test_03_addresses.py
- test_04_products.py
- test_05_cart.py
- test_06_coupons.py
- test_07_checkout.py
- test_08_wallet.py
- test_09_loyalty.py
- test_10_orders.py
- test_11_reviews.py
- test_12_support.py
- test_13_additional_bugs.py

1.2 Category Coverage Summary

Category	What was validated	Why it is important

Category	What was validated	Why it is important
Valid requests	Happy paths for admin, profile, products, checkout, wallet, loyalty, orders, support	Confirms API works for correct inputs and returns proper JSON structure
Invalid inputs	Out-of-range values (rating, quantity), invalid payment/status values	Ensures robust validation and prevents invalid state transitions
Missing fields/headers	Missing `X-Roll-Number`, missing `X-User-ID`, missing body fields	Verifies authentication/authorization and request contract enforcement
Wrong data types	String/float where integer expected, malformed JSON-like payload shapes	Prevents silent corruption and ambiguous behavior
Boundary values	Name length, phone length, quantity 0/1, amount 0/100001, comment length, status transitions	Catches off-by-one and edge-condition defects

1.3 Execution Results

Command used:

```
pytest -q dass_testing/blackbox/tests
```

Observed result:

- Total tests: `51`
- Passed: `28`
- Failed assertions: `23`
- Bug test cases identified: `23`
- Unique root-cause bugs: `20`

1.4 Detailed Test Case Matrix (All 51 Cases)

Each test case includes input, expected output, and justification/importance.

#	Test Case	Input	Expected Output	Justification / Importance
1	`test_admin_users_missing_roll_header_returns_401`	`GET /admin/users without X-Roll-Number`	`401`	Verifies mandatory global auth header enforcement.

#	Test Case	Input	Expected Output	Justification / Importance
2	`test_admin_users_invalid_roll_header_returns_400`	`GET /admin/users with X-Roll-Number: abc`	`400`	Validates roll-number type checking.
3	`test_admin_users_success_structure`	`GET /admin/users with valid roll header`	`200 and user list schema`	Baseline admin endpoint correctness.
4	`test_profile_requires_user_id_header`	`GET /profile without X-User-ID`	`400`	Confirms user-scoped header requirement.
5	`test_profile_update_rejects_short_name`	`PUT /profile with 1-char name`	`400`	Validates minimum name length boundary.
6	`test_profile_update_rejects_non_digit_phone`	`PUT /profile with alphanumeric phone`	`400`	Validates phone digits-only rule.
7	`test_profile_update_valid_boundary`	`PUT /profile with 2-char name and 10-digit phone`	`200`	Checks valid lower-boundary acceptance.
8	`test_address_create_invalid_label_rejected`	`POST /addresses with label=home`	`400`	Validates enum constraint (HOME/OFFICE/OTHER).
9	`test_address_create_invalid_pincode_wrong_length_rejected`	`POST /addresses with 5-digit pincode`	`400`	Validates exact-length pincode boundary.
10	`test_address_update_response_must_return_updated_data`	`Create and update address street/default`	`200 with updated response fields`	Ensures update responses are not stale.
11	`test_products_list_shows_only_active`	`GET /products`	`200 and all is_active=true`	Validates active-only listing rule.
12	`test_get_missing_product_returns_404`	`GET /products/999999`	`404`	Validates missing-resource behavior.
13	`test_products_sort_price_ascending`	`GET /products?sort=price_asc`	`Prices sorted ascending`	Validates sorting feature correctness.

#	Test Case	Input	Expected Output	Justification / Importance
14	`test_cart_add_quantity_zero_must_be_rejected`	`POST /cart/add quantity=0`	`400`	Checks quantity lower-bound rejection.
15	`test_cart_add_negative_quantity_rejected`	`POST /cart/add quantity=-1`	`400`	Checks negative quantity rejection.
16	`test_cart_add_missing_product_404`	`POST /cart/add with non-existing product`	`404`	Validates product existence check.
17	`test_cart_subtotal_and_total_must_match_quantity_times_price`	`Add item then GET /cart`	`subtotal=qty*price and total=sum(subtotals)`	Validates core financial arithmetic.
18	`test_coupon_expired_rejected`	`POST /coupon/apply with expired coupon`	`400`	Validates coupon expiry enforcement.
19	`test_coupon_fixed_discount_applies_correctly`	`Apply fixed coupon on known subtotal`	`Exact fixed discount and new total`	Validates fixed discount computation.
20	`test_coupon_percent_discount_must_use_percentage`	`Apply percent coupon FIRSTORDER`	`Percent discount with cap`	Validates percent coupon logic.
21	`test_checkout_rejects_invalid_payment_method`	`POST /checkout payment_method=UPI`	`400`	Validates payment-method domain constraints.
22	`test_checkout_rejects_empty_cart`	`POST /checkout on empty cart`	`400`	Validates non-empty cart precondition.
23	`test_checkout_card_payment_status_paid`	`Valid CARD checkout`	`200 and payment_status=PAID`	Validates payment-status semantics.
24	`test_wallet_get_structure`	`GET /wallet`	`200 with wallet_balance`	Validates wallet response schema.
25	`test_wallet_add_rejects_zero_and_above_max`	`POST /wallet/add amount=0 and 100001`	`400`	Validates top-up boundary constraints.
26	`test_wallet_pay_insufficient_balance_rejected`	`POST /wallet/pay very high amount`	`400`	Validates insufficient-funds guard.

#	Test Case	Input	Expected Output	Justification / Importance
27	`test_loyalty_get_structure`	`GET /loyalty`	`200 with loyalty_points`	Validates loyalty response schema.
28	`test_loyalty_redeem_minimum_points_boundary`	`POST /loyalty/redeem points=0`	`400`	Validates minimum redeemable points.
29	`test_orders_list_structure`	`GET /orders`	`200 list structure`	Validates order listing contract.
30	`test_cancel_non_existent_order_returns_404`	`POST /orders/99999999/cancel`	`404`	Validates cancellation not-found handling.
31	`test_review_rating_out_of_range_rejected`	`POST /products/1/reviews rating=6`	`400`	Validates rating upper boundary.
32	`test_review_comment_empty_rejected`	`POST /products/1/reviews empty comment`	`400`	Validates comment lower boundary.
33	`test_review_average_must_be_decimal_not_truncated`	`GET /products/1/reviews`	`Exact decimal average`	Validates precision of aggregate rating.
34	`test_support_ticket_subject_too_short_rejected`	`POST /support/ticket with 4-char subject`	`400`	Validates subject minimum length.
35	`test_support_ticket_initial_status_open`	`Valid support ticket creation`	`200 with initial OPEN status`	Validates initial workflow state.
36	`test_support_status_transition_must_be_one_way`	`OPEN ticket updated directly to CLOSED`	`400`	Validates one-way transition policy.
37	`test_profile_nonexistent_user_header_should_be_400`	`GET /profile with non-existing user id`	`400`	Validates invalid user-scoped identity handling.
38	`test_address_pincode_non_digit_should_be_rejected`	`POST /addresses with alphanumeric pincode`	`400`	Validates digit-only pincode rule.

#	Test Case	Input	Expected Output	Justification / Importance
39	test_cart_update_non_existing_product_should_404	POST /cart/update non-existing product	404	Validates update-target existence.
40	test_cart_subtotal_exact_after_update_quantity	Add then update cart quantity	Exact recomputed subtotal	Validates arithmetic consistency after mutation.
41	test_cart_total_single_item_must_equal_subtotal	One-item cart then GET /cart	Total equals item subtotal	Validates minimal-cart aggregation.
42	test_cart_total_multiple_items_must_include_last_item	Two-item cart then GET /cart	Total includes all item subtotals	Detects missing-item summation bugs.
43	test_checkout_empty_cart_card_rejected	Empty cart checkout with CARD (user 66)	400	Confirms empty-cart guard in alternate user context.
44	test_checkout_empty_cart_cod_rejected	Empty cart checkout with COD (user 66)	400	Validates empty-cart guard across payment methods.
45	test_checkout_empty_cart_wallet_rejected	Empty cart checkout with WALLET (user 66)	400	Validates empty-cart guard across payment methods.
46	test_wallet_pay_should_deduct_exact_amount	Wallet pay fixed amount then compare balances	Exact deduction only	Validates no over/under debit in money flow.
47	test_reviews_average_should_keep_decimal_precision	GET /products/1/reviews and recompute mean	Exact decimal match	Confirms precision in aggregate metrics.
48	test_support_invalid_status_value_rejected	Update ticket with status=REOPENED	400	Validates support status enum enforcement.
49	test_support_open_to_closed_direct_transition_rejected	OPEN -> CLOSED transition	400	Validates forbidden transition path.
50	test_support_closed_to_in_progress_transition_rejected	CLOSED -> IN PROGRESS transition	400	Validates backward transition rejection.

#	Test Case	Input	Expected Output	Justification / Importance
51	`test_support_in_progress_to_open_transition_rejected`	`IN PROGRESS -> OPEN transition`	`400`	Validates backward transition rejection.

2. Coverage Attempt

Command used:

```
coverage run -m pytest -q dass_testing/blackbox/tests || true
coverage report -m
```

Observed coverage (test code): `99%`

Note: This is black-box testing against a Docker image; internal API implementation coverage cannot be measured from outside.

3. Detailed Bug Reports (23 Failing Bug Test Cases)

All requests use `Content-Type: application/json` where request body is present.

Bug 1: Profile accepts non-digit phone

- Failing test case number(s): #6
- Endpoint tested: `PUT /api/v1/profile`
- Request payload:
  - Method: `PUT`
  - URL: `http://localhost:8080/api/v1/profile`
  - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 8`
  - Body: `{"name":"Valid Name","phone":"12345abcde"}`
- Expected result: `400` (phone must be exactly 10 digits)
- Actual result: `200` with `{"message":"Profile updated successfully"}`

Bug 2: Invalid user id (non-existing) returns 404 instead of documented 400

- Failing test case number(s): #37

- Endpoint tested: `GET /api/v1/profile`
- Request payload:
  - Method: `GET`
  - URL: `http://localhost:8080/api/v1/profile`
  - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 999999`
  - Body: `N/A`
- Expected result: `400` (invalid user-scoped header)
- Actual result: `404` with `{"error":"User not found"}`

Bug 3: Address update response returns stale data

- Failing test case number(s): #10
- Endpoint tested: `PUT /api/v1/addresses/{address_id}`
- Request payload:
  - Method: `PUT`
  - URL: `http://localhost:8080/api/v1/addresses/<created_id>`
  - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 5`
  - Body: `{"street":"11111 New Street","is_default":true}`
- Expected result: Updated response object must show new street
- Actual result: Response still shows old street

Bug 4: Address accepts non-digit pincode

- Failing test case number(s): #38
- Endpoint tested: `POST /api/v1/addresses`
- Request payload:
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/addresses`
  - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 5`
  - Body: `{"label":"HOME","street":"12345 Main Street","city":"Hyd","pincode":"ABCDE1","is_default":false}`
- Expected result: `400` (pincode must be 6 digits)
- Actual result: `200` address created successfully

Bug 5: Cart add accepts quantity 0

- Failing test case number(s): #14
- Endpoint tested: `POST /api/v1/cart/add`

- Request payload:
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/cart/add`
  - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 2`
  - Body: `{"product_id":1,"quantity":0}`
- Expected result: `400`
- Actual result: `200` (`Item added to cart`)

Bug 6: Cart update on missing product returns 200

- Failing test case number(s): #39
- Endpoint tested: `POST /api/v1/cart/update`
- Request payload:
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/cart/update`
  - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 2`
  - Body: `{"product_id":999999,"quantity":1}`
- Expected result: `404`
- Actual result: `200` (`Cart updated successfully`)

Bug 7: Cart subtotal arithmetic incorrect (add flow)

- Failing test case number(s): #17
- Endpoint tested: `GET /api/v1/cart`
- Request payload:
  - Method: `POST` then `GET`
  - URL: `http://localhost:8080/api/v1/cart/add` then `http://localhost:8080/api/v1/cart`
  - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 2`
  - Body (add): `{"product_id":1,"quantity":3}`
- Expected result: subtotal `3 * 120 = 360`
- Actual result: subtotal `104`

Bug 8: Cart subtotal arithmetic incorrect (update flow)

- Failing test case number(s): #40
- Endpoint tested: `GET /api/v1/cart`
- Request payload:

    - Method: `POST` update then `GET`
    - URL: `http://localhost:8080/api/v1/cart/update` then `http://localhost:8080/api/v1/cart`
    - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 2`
    - Body (update): `{"product_id":1,"quantity":4}`
- Expected result: subtotal `4 * 120 = 480`
- Actual result: subtotal `-32`

Bug 9: Cart total incorrect for single-item cart

- Failing test case number(s): #41
- Endpoint tested: `GET /api/v1/cart`
- Request payload:
    - Method: `POST` add then `GET`
    - URL: `http://localhost:8080/api/v1/cart/add` then `http://localhost:8080/api/v1/cart`
    - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 2`
    - Body (add): `{"product_id":2,"quantity":2}`
- Expected result: total equals that item subtotal
- Actual result: total `0`, subtotal `4`

Bug 10: Cart total incorrect for multi-item cart

- Failing test case number(s): #42
- Endpoint tested: `GET /api/v1/cart`
- Request payload:
    - Method: two `POST /cart/add` then `GET /cart`
    - URL: `http://localhost:8080/api/v1/cart`
    - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 2`
    - Bodies: `{"product_id":1,"quantity":2}` and `{"product_id":2,"quantity":2}`
- Expected result: total equals sum of all subtotals
- Actual result: mismatch (example: total `-16`, subtotal sum `-12`)

Bug 11: Percentage coupon applied as fixed amount

- Failing test case number(s): #20
- Endpoint tested: `POST /api/v1/coupon/apply`

- Request payload:
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/coupon/apply`
  - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 2`
  - Body: `{"coupon_code":"FIRSTORDER"}`
  - Cart setup: subtotal `1000`
- Expected result: discount `150`, new total `850`
- Actual result: discount `15`, new total `985`

Bug 12: Checkout allows empty cart with CARD

- Failing test case number(s): #43
- Endpoint tested: `POST /api/v1/checkout`
- Request payload:
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/checkout`
  - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 66`
  - Body: `{"payment_method":"CARD"}`
- Expected result: `400`
- Actual result: `200`, order created with `total_amount: 0`

Bug 13: Checkout allows empty cart with COD

- Failing test case number(s): #44
- Endpoint tested: `POST /api/v1/checkout`
- Request payload:
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/checkout`
  - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 66`
  - Body: `{"payment_method":"COD"}`
- Expected result: `400`
- Actual result: `200`, order created with `total_amount: 0`

Bug 14: Checkout allows empty cart with WALLET

- Failing test case number(s): #45
- Endpoint tested: `POST /api/v1/checkout`
- Request payload:

    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/checkout`
    - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 66`
    - Body: `{"payment_method":"WALLET"}`
- Expected result: `400`
- Actual result: `200`, order created with `total_amount: 0`

Bug 15: Wallet pay does not deduct exact amount

- Failing test case number(s): #46
- Endpoint tested: `POST /api/v1/wallet/pay`
- Request payload:
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/wallet/pay`
    - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 9`
    - Body: `{"amount":10}`
- Expected result: balance decreases by exactly `10`
- Actual result: balance decreased by `10.4`

Bug 16: Review average is truncated to integer

- Failing test case number(s): #33
- Endpoint tested: `GET /api/v1/products/1/reviews`
- Request payload:
    - Method: `GET`
    - URL: `http://localhost:8080/api/v1/products/1/reviews`
    - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 12`
    - Body: `N/A`
- Expected result: exact decimal mean of ratings
- Actual result: integer average (e.g., `3` instead of `3.3333...`)

Bug 17: Support allows invalid status value

- Failing test case number(s): #48
- Endpoint tested: `PUT /api/v1/support/tickets/{ticket_id}`
- Request payload:
    - Method: `PUT`
    - URL: `http://localhost:8080/api/v1/support/tickets/<created_id>`

    - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 13`
    - Body: `{"status":"REOPENED"}`
- Expected result: `400`
- Actual result: `200` and status updated

Bug 18: Support allows OPEN -> CLOSED directly

- Failing test case number(s): #49
- Endpoint tested: `PUT /api/v1/support/tickets/{ticket_id}`
- Request payload:
    - Method: `PUT`
    - URL: `http://localhost:8080/api/v1/support/tickets/<created_id>`
    - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 13`
    - Body: `{"status":"CLOSED"}`
- Expected result: `400`
- Actual result: `200`

Bug 19: Support allows CLOSED -> IN_PROGRESS reversal

- Failing test case number(s): #50
- Endpoint tested: `PUT /api/v1/support/tickets/{ticket_id}`
- Request payload:
    - Method: `PUT`
    - URL: `http://localhost:8080/api/v1/support/tickets/<created_id>`
    - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 13`
    - Body: `{"status":"IN_PROGRESS"}` after ticket already reached `CLOSED`
- Expected result: `400`
- Actual result: `200`

Bug 20: Support allows IN_PROGRESS -> OPEN reversal

- Failing test case number(s): #51
- Endpoint tested: `PUT /api/v1/support/tickets/{ticket_id}`
- Request payload:
    - Method: `PUT`
    - URL: `http://localhost:8080/api/v1/support/tickets/<created_id>`
    - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 13`
    - Body: `{"status":"OPEN"}` after ticket reached `IN_PROGRESS`

- Expected result: `400`
- Actual result: `200`

Bug 21: Duplicate confirmation of empty-cart checkout acceptance (separate failing test)

- Failing test case number(s): #22
- Endpoint tested: `POST /api/v1/checkout`
- Request payload:
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/checkout`
  - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 7`
  - Body: `{"payment_method":"CARD"}`
- Expected result: `400` (cart must not be empty)
- Actual result: `200`, zero-total order created

Bug 22: Duplicate confirmation of support OPEN -> CLOSED transition violation (separate failing test)

- Failing test case number(s): #36
- Endpoint tested: `PUT /api/v1/support/tickets/{ticket_id}`
- Request payload:
  - Method: `PUT`
  - URL: `http://localhost:8080/api/v1/support/tickets/<created_id>`
  - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 13`
  - Body: `{"status":"CLOSED"}`
- Expected result: `400` (must go OPEN -> IN_PROGRESS first)
- Actual result: `200`

Bug 23: Duplicate confirmation of review average precision failure (separate failing test)

- Failing test case number(s): #47
- Endpoint tested: `GET /api/v1/products/1/reviews`
- Request payload:
  - Method: `GET`
  - URL: `http://localhost:8080/api/v1/products/1/reviews`

    - Headers: `X-Roll-Number: 2024101096`, `X-User-ID: 12`
    - Body: `N/A`
- Expected result: decimal average exactly matches arithmetic mean
- Actual result: truncated integer average